

InsightPilot: An Agentic Platform for Conversational Business Intelligence

Project Workbook

By

Akshay Sunil Navani
Atharva Nitin Kulkarni
Pranjal Shrivastava
Shantanu Zadbuke

October 8, 2026

Advisor: Professor Vijay Eranti

Project summary. InsightPilot is a conversational business intelligence platform for enterprise data analysts. An orchestrator turns a business question into an *analysis plan*, an ordered set of tasks over named data sources, and shows it to the analyst for approval before anything runs. Specialist agents then carry out the plan: data fetch and analytics, with a visualization agent added in CMPE 295B. Each handoff between agents is a *hop*, and every hop records its *lineage*: the sources used, the query issued, the transform applied and the row counts in and out. Each hop is checked by a separate, simpler query, not by asking the model whether it is confident, so the analyst can inspect the one hop they doubt instead of redoing the whole analysis. The unit of work is the *investigation*: one business question taken to a verified answer. The project repository is https://github.com/akshaynavani/CMPE295A_InsightPilot.

Team responsibilities. Each member leads one technical area across both semesters and wrote the workbook sections listed against their name.

Table 1. Team responsibilities and section ownership

Member	Primary technical ownership	Workbook sections owned
Atharva Kulkarni	Orchestration tier: semantic resolver, plan synthesiser, orchestrator and replanner, model client	Literature Search (planning and agents), Ch. 3 requirements, Ch. 8 prototyping environment and sample programs
Akshay Navani	Integration tier: Postgres and CSV/Excel connectors, application store, packaging	Literature Search (text-to-SQL), Ch. 5 architecture, Ch. 8 programming environment and development tools
Pranjal Shrivastava	Verification and evaluation: hop verifier, cross-source reconciler, evaluation harness	Literature Search (reliability and evaluation), Ch. 4 dependencies, Ch. 8 prototype implementation plan, Ch. 9 schedule
Shantanu Zadbukeye	Presentation and delivery: investigation workspace, plan review, lineage inspector and export, audit log	State-of-the-Art Summary, Ch. 2 justification, Ch. 6 and Ch. 7 planning

Chapter 1. Literature Search, State of the Art

Literature Search

The literature relevant to InsightPilot falls into four areas: natural-language-to-SQL generation over realistic schemas, multi-step planning by LLM agents, agent reliability across chained steps, and automated machine learning on tabular data.

Natural language to SQL. Accuracy is high on clean benchmarks and falls sharply on realistic ones. On Spider 2.0, top systems reach 96.70% execution accuracy on the Snow setting, 76.23% on Lite and 65.60% on the DBT setting [1]. The 31-point drop comes from adding semantic-layer indirection and real project structure. On BIRD, with 12,751 question–SQL pairs across 95 databases, the best single model scores 80.04% against 92.96% for humans [2]. Annotation errors in these benchmarks are common enough that gaps of a few points between leaderboard entries are not meaningful [3], so this project measures accuracy on its own fixed question set instead of citing leaderboard differences. The errors that remain are often silent. A study of 4,602 incorrect queries from in-context-learning text-to-SQL techniques found that schema errors, such as referencing a column that does not exist, cause 81.2% of the queries that fail to execute, so the database itself reports them. Semantic errors are different: 30.9% of all errors were queries that ran successfully but misread the question or the schema, returning a plausible but wrong result [4]. That silent failure mode shapes this project's design. Arctic-Text2SQL-R1 shows that training on execution feedback lets small models compete with much larger ones [5]. This project applies the same signal at inference time: database errors are returned to the generator in a query repair loop.

Multi-step planning and data reasoning. The orchestrator depends on multi-step analytical reasoning, the weakest capability measured in the literature. DABstep [6] contains 450 tasks drawn from a real financial-analytics workload, 84% of them classed as Hard. The best agent scores 14.55% on the Hard tasks against 76.39% on the Easy ones, and the authors describe multi-step tasks as largely unsolved. A survey of evaluation tools for data-science assistants and agents [7] finds that existing benchmarks focus on a small set of goal-oriented activities and largely ignore data management and exploratory analysis.

Agent reliability and error propagation. First, consistency is lower than average accuracy suggests. τ -bench [8] introduces pass^k , the probability that an agent solves the same task in all k trials. Its best function-calling agent succeeds on 61.2% of retail tasks in a single trial, but its pass^8 falls below 25%. An analyst who asks the same question twice and gets two different answers cannot rely on either. Second, errors compound across steps. If each step were independently 95% accurate, a ten-step chain would be correct about 60% of the time, and at 90% per step about 35%. Two failure studies show where this happens. A taxonomy of agent failures across memory, reflection, planning and action modules finds that a single root-cause error propagating through later steps is the main cause of task failure [9]. A diagnostic study of tool

invocation finds tool-initialisation failure to be the leading error, at an 89% error rate for a 3-billion-parameter model and absent in large models [10]. That result bounds the deployment design: the orchestration layer must call a frontier-class model rather than a small local one.

Agent task horizon. METR measures the length of task, in human working time, that agents complete with 50% reliability. That horizon has doubled about every seven months over six years [11], and about every three months since 2024 [12]. This is the strongest evidence that a system like InsightPilot is now worth attempting. The horizon is measured at 50% reliability, though, so it shows a trend rather than readiness: an analytical answer that is right half the time has no value.

Automated machine learning on tabular data. MLE-bench evaluates agents on 75 Kaggle competitions, each scored by its original metric [13]. AutoMLGen reaches a 36.4% average medal rate, with 18.7% gold, under a 12-hour budget [14]. Conversational products have begun to expose this capability: Qlik's Predict agent trains regression and classification models from natural-language requests inside Qlik Answers [15]. A conversational workflow cannot contain a 12-hour step, however, so any modelling step in this project therefore has to be time-boxed in minutes and must report what it did not attempt.

Research gap. These areas are mostly studied in isolation, and the commercial systems in the next section secure correctness by requiring a human-authored semantic model before a question is asked. Recent systems cover parts of what this project needs. Data Interpreter [16] and AgenticData [17] plan and refine multi-step analyses over heterogeneous data. RADAR [18] records each operation's inputs and observations, rejects operations that conflict with them, and measures silent errors directly: across 1,054 tasks it reduces them from 405 to 289 against the strongest baseline. Databricks Genie Inspect checks a generated query by writing smaller verification queries, but within a single query rather than across a plan [20]. We found no system that verifies every hop of a multi-source plan with an independent query, structures its lineage so that checking an answer costs less than re-deriving it, and reports multi-hop execution accuracy alongside a silent-error rate. InsightPilot builds and measures such a system for enterprise data analysts.

State-of-the-Art Summary

Commercial conversational BI products share one correctness mechanism: they answer inside a scope that someone has curated in advance. Snowflake Cortex Analyst generates SQL against a semantic view, which can be written by hand or drafted with Semantic View Autopilot [19]. Databricks AI/BI Genie answers over a Genie Agent (formerly a Genie space) of up to 50 tables, views or metric views [20]. Power BI Copilot answers from one semantic model and can generate ad hoc DAX for calculations the model does not contain, but it cannot forecast or detect anomalies [21]. Tableau Pulse is built around pre-defined metrics: it pushes changes to users and, with Tableau+, answers

questions only within those metrics [22]. Enterprise budgets follow this approach. In a survey of 818 organisations with more than \$100M in revenue, nearly 59% are directing new budget to semantic layers, and 24.9% name accuracy and hallucination risk as their top concern about generative AI in analytics [23].

The approach is sound: a curated semantic layer prevents many of the semantic errors described in the Literature Search, where a query runs but misreads the schema [4]. Its limits are moving outward, through automated drafting, larger table limits and ad hoc calculations, but the scope is still prepared before the question is asked. A question that needs data nobody has added to the model cannot be answered until someone adds it.

First, verification is reaching products. Genie's Inspect, in Public Preview since March 2026, reviews the SQL Genie generated, runs smaller queries to verify specific parts of it, and generates improved SQL as needed [20]. Power BI Copilot with Fabric IQ shows its reasoning steps, the DAX query and the data behind each answer [21]. Both check or expose one answer within one curated scope; neither verifies each step of a plan that spans sources. Second, the Model Context Protocol, released by Anthropic in November 2024 [33] and contributed to the Linux Foundation's Agentic AI Foundation in December 2025 [34], standardises how agents access tools but not what the data means [24]. It makes a pluggable connector layer practical for a small team but leaves schema understanding unsolved.

The category also has a record of failure. IBM discontinued Watson Analytics in 2019 [25]. Narrative Science was acquired by Salesforce and folded into Tableau in 2021 [26]. Sisu Data raised about \$128.7M for automated diagnosis and joined Snowflake in 2023 [27]. Power BI Q&A is being fully retired in December 2026 [28]. None of these failed for lack of distribution.

Table 2. Leading industry offerings compared with this project

Offering	Primary customer	What it provides	Gap for the enterprise analyst
Snowflake Cortex Analyst [19]	Snowflake enterprises	Text-to-SQL over a semantic view, written by hand or drafted with Semantic View Autopilot	Answers only over Snowflake data described in a semantic view
Databricks AI/BI Genie [20]	Lakehouse enterprises	Conversational SQL over a Genie Agent of up to 50 tables; Inspect verifies its own generated SQL	Scope fixed by the Genie Agent's tables; verification covers one query, not a multi-step plan
Power BI Copilot [21]	Fabric customers	Natural language over a Power BI semantic model, including ad hoc DAX; Fabric IQ shows reasoning steps and the DAX used	Answers only within one semantic model; no forecasting or anomaly detection
Tableau Pulse [22]	Tableau Cloud customers	Metric monitoring with natural-language summaries; Q&A on Tableau+	Limited to insights from pre-defined Pulse metrics
Wren AI [29]	Open-source GenBI users	Open-source agent with its own modelling language (MDL); about 18,000 GitHub stars; dry-plan SQL validation	Requires an MDL model of the data; validation is per query
Hex Notebook Agent [30]	Analysts working in notebooks	Agent in a SQL and Python notebook that can draft a plan and run multi-step work, showing each change as a diff	A plan is optional guidance, not an enforced structure; the analyst reviews each change
This project	Enterprise data analysts	A plan approved before it runs, executed across Postgres and CSV/Excel files (REST APIs in CMPE 295B), with every hop checked by an independent query and recorded as lineage	Not applicable

Leading-edge techniques adopted: hybrid schema linking with reciprocal rank fusion and margin-based ambiguity detection; constrained generation into a typed plan DAG with static validation before execution; verification by decomposition in the style of Genie's Inspect [20] but applied across a multi-source plan; execution-feedback query repair [5]; consumption of an existing dbt semantic layer when present rather than requiring a new one; and repeated-trial reliability measurement using `pass^k` [8].

Published accuracy. None of the major BI platforms reviewed here publishes product-level execution accuracy on BIRD or Spider 2.0. Vendors publish scores for their models, such as Arctic-Text2SQL-R1 [5], and specialist data-agent companies post product results on the Spider 2.0 leaderboard [1]. This project therefore makes no accuracy comparison with an incumbent and reports its own figures on its own question set.

References

1. Lei, F., Chen, J., Ye, Y., Cao, R., Shin, D., Su, H., Suo, Z., Gao, H., Hu, W., Yin, P., Zhong, V., Xiong, C., Sun, R., Liu, Q., Wang, S., & Yu, T. (2025). Spider 2.0: Evaluating language models on real-world enterprise text-to-SQL workflows. In *International Conference on Learning Representations (ICLR)*. arXiv:2411.07763

Peer-reviewed benchmark for enterprise text-to-SQL workflows. The Snow, Lite and DBT accuracy figures quoted are read from the project's public leaderboard, <https://spider2-sql.github.io/>, accessed October 2026.

2. Li, J., Hui, B., Qu, G., Yang, J., Li, B., Li, B., Wang, B., Qin, B., Cao, R., Geng, R., Huo, N., Zhou, X., Ma, C., Li, G., Chang, K. C. C., Huang, F., Cheng, R., & Li, Y. (2023). Can LLM already serve as a database interface? A big bench for large-scale database grounded text-to-SQLs. In *Advances in Neural Information Processing Systems (NeurIPS)* 36. arXiv:2305.03111

The BIRD benchmark: 12,751 question-SQL pairs across 95 databases, with the human-performance baseline this workbook compares against.

3. Jin, T., Choi, Y., Zhu, Y., & Kang, D. (2026). Text-to-SQL benchmarks are broken: An in-depth analysis of annotation errors. In *Conference on Innovative Data Systems Research (CIDR '26)*, January 18-21, 2026, Chaminade, USA.

Peer-reviewed analysis establishing that small leaderboard differences are not meaningful; the reason this project measures its own fixed question set.

4. Shen, J., Wan, C., Qiao, R., Zou, J., Xu, H., Shao, Y., Zhang, Y., Miao, W., & Pu, G. (2026). Understanding, detecting, and repairing real-world in-context-learning-based text-to-SQL errors. *Proceedings of the ACM on Software Engineering*, 3(FSE), Article FSE164. arXiv:2501.09310

Peer-reviewed study of 4,602 incorrect queries; source of the schema-error (81.2% of execution failures) and semantic-error (30.9% of all errors) figures.

5. Yao, Z., Sun, G., Borchmann, L., Shen, Z., Deng, M., Zhai, B., Zhang, H., Li, A., & He, Y. (2026). Arctic-Text2SQL-R1: Simple rewards, strong reasoning in text-to-SQL. In *Findings of the Association for Computational Linguistics: ACL 2026* (pp. 26966–26995). <https://doi.org/10.18653/v1/2026.findings-acl.1345>

Execution-feedback reinforcement learning for SQL generation; the basis for this project's query repair loop.

6. Egg, A., Iglesias Goyanes, M., Kingma, F., Mora, A., von Werra, L., & Wolf, T. (2025). DABstep: Data agent benchmark for multi-step reasoning. arXiv:2506.23719

450+ real analytics tasks; the ~14.55-16% Hard-split result that is this project's central risk expressed as a benchmark.

7. Testini, I., Pacchiardi, L., & Hernández-Orallo, J. (2025). Measuring data science automation: A survey of evaluation tools for AI assistants and agents. *Transactions on Machine Learning Research*. <https://openreview.net/forum?id=MB0TCLfLn1>

Survey of the data-science-agent evaluation landscape; shows which capabilities have benchmarks and which do not.

8. Yao, S., Shinn, N., Razavi, P., & Narasimhan, K. (2025). τ -bench: A benchmark for tool-agent-user interaction in real-world domains. In *International Conference on Learning Representations (ICLR)*. arXiv:2406.12045

Introduces the pass^k reliability metric and the collapse from above 60% pass¹ to below 25% pass⁸.

9. Zhu, K., Tian, M., Li, B., Liu, Z., Yang, Y., Zhang, J., Han, P., Xie, Q., Cui, F., Zhang, W., Ma, X., Yu, X., Ramesh, G., Wu, J., Tang, R., Liu, Z., Ji, H., Lu, P., Zou, J., & You, J. (2026). AgentDebug: Where LLM agents fail and how they can learn from failures. In *ICML 2026 Workshop on Failure Modes in Agentic AI*. arXiv:2509.25370

Agent error taxonomy across memory, reflection, planning, action and system levels, identifying error propagation as the primary reliability bottleneck.

10. Huang, D., Malwe, G., & Wang, Z. (2026). When agents fail to act: A diagnostic framework for tool invocation reliability in multi-agent LLM systems. In *9th International Conference on Artificial Intelligence and Big Data (ICAIBD)*. arXiv:2601.16280

Twelve-category error taxonomy; the 89% tool-initialisation error rate in small models that bounds this project's deployment model.

11. METR. (2025, March 19). Measuring AI ability to complete long tasks. <https://metr.org/blog/2025-03-19-measuring-ai-ability-to-complete-long-tasks/>

Task-horizon doubling approximately every seven months; research-organisation report rather than a peer-reviewed paper.

12. METR. (2026, January 29). Time horizon 1.1. <https://metr.org/blog/2026-1-29-time-horizon-1-1/>

Updated horizon estimate placing the doubling time since 2024 at 89 days, about three months.

13. Chan, J. S., Chowdhury, N., Jaffe, O., Aung, J., Sherburn, D., Mays, E., Starace, G., Liu, K., Maksin, L., Patwardhan, T., Mađry, A., & Weng, L. (2025). MLE-bench: Evaluating machine learning agents on machine learning engineering. In *International Conference on Learning Representations (ICLR)*. arXiv:2410.07095

75 Kaggle competitions scored by original competition metrics; the standard harness for autonomous ML engineering.

14. Du, S., Yan, X., Jiang, D., Yuan, J., Hu, Y., Li, X., He, L., Zhang, B., & Bai, L. (2025). AutoMLGen: Navigating fine-grained optimization for coding agents. arXiv:2510.08511

36.4% medal rate and 18.7% gold on MLE-bench under a 12-hour budget; the result that forces a minutes-long time box.

15. Qlik. Predict agent in Qlik Answers. Qlik Cloud help. https://help.qlik.com/en-US/cloud-services/Subsystems/Hub/Content/Sense_Hub/QlikAnswers/predict-agent.htm

Vendor documentation of natural-language training of regression and classification models inside a conversational BI product; time-series models are not supported.

16. Hong, S., Lin, Y., Liu, B., Liu, B., Wu, B., Zhang, C., Li, D., Chen, J., Zhang, J., Wang, J., Zhang, L., Zhang, L., Yang, M., Zhuge, M., Guo, T., Zhou, T., Tao, W., Tang, R., Lu, X., Zheng, X., Liang, X., Fei, Y., Cheng, Y., Ni, Y., Gou, Z., Xu, Z., Luo, Y., & Wu, C. (2025). Data Interpreter: An LLM agent for data science. In *Findings of the Association for Computational Linguistics: ACL 2025* (pp. 19796–19821). <https://doi.org/10.18653/v1/2025.findings-acl.1016>

Hierarchical, graph-based planning with step-by-step verification and refinement for data-science workflows.

17. Sun, J., Li, G., Zhou, P., Ma, Y., Xu, J., & Li, Y. (2025). AgenticData: An agentic data analytics system for heterogeneous data. arXiv:2508.05002

Multi-agent analytics over structured and unstructured sources with iterative verification and refinement of the analytics plan.

18. Yan, H., Deng, L., Wang, Z., Wang, Y., & Liu, C. (2026). Fail loudly: An auditable runtime for agentic data analysis. arXiv:2609.32528

RADAR: records typed operations and observations, validates operations against them at runtime, and reports silent errors across KramaBench, DA-Code and DABstep. The closest prior work to this project.

19. Snowflake. Cortex Analyst documentation. <https://docs.snowflake.com/en/user-guide/snowflake-cortex/cortex-analyst> (accessed October 2026)

Vendor documentation: semantic views are the recommended approach, created by hand or with Semantic View Autopilot; legacy YAML semantic models remain supported.

20. Databricks. AI/BI and Genie release notes 2026. <https://docs.databricks.com/aws/en/ai-bi/release-notes/2026> (accessed October 2026)

Release notes recording the 50-table Genie Agent limit (September 2026), the renaming of Genie spaces to Genie Agents, and Inspect in Public Preview (March 2026).

21. Microsoft. (2026). Ask data questions with Copilot and Fabric IQ - Power BI. Microsoft Learn. <https://learn.microsoft.com/en-us/power-bi/create-reports/copilot-ask-data-question>

Primary documentation of ad hoc DAX calculations, Fabric IQ reasoning steps, and unsupported forecasting and anomaly detection.

22. Tableau. Ask questions and discover insights in Tableau Pulse. Tableau Help. https://help.tableau.com/current/online/en-us/pulse_ask_discover_qa.htm

Vendor documentation: Tableau Agent in Pulse is a Tableau+ feature limited to insights from Pulse metrics.

23. Futurum Group. (2026). Enterprise data analytics survey (n=818). <https://futurumgroup.com/press-release/enterprise-data-analytics-survey-finds-59-investing-in-semantic-layers-as-critical-ai-infrastructure/>

Named survey with disclosed sample; the nearly 59% semantic-layer investment and 24.9% accuracy-reservation figures.

24. Model Context Protocol. (2025). *Specification, version 2025-11-25*. <https://modelcontextprotocol.io/specification/2025-11-25>

The authoritative protocol definition; the basis for the claim that MCP standardises tool access rather than meaning.

25. IBM Community. (2019). IBM Watson Analytics free edition trial has expired. <https://community.ibm.com/community/user/discussion/ibm-watson-analytics-free-edition-trial-has-expired>

Record of the discontinuation of the flagship natural-language BI product of its generation.

26. TechTarget. (2021). Tableau completes acquisition of Narrative Science.
<https://www.techtarget.com/searchbusinessanalytics/news/252511067/Tableau-completes-acquisition-of-Narrative-Science>

Records the 2021 acquisition of Narrative Science by Salesforce and its integration into Tableau.

27. Crunchbase. Snowflake acquires Sisu Data.
<https://www.crunchbase.com/acquisition/snowflake-computing-acquires-sisu-data--ea3bc847>

Records approximately \$128.7M raised before acquisition by the warehouse vendor.

28. Microsoft Fabric. (2026). Deprecating Power BI Q&A.
<https://community.fabric.microsoft.com/t5/Power-BI-Updates-Blog/Deprecating-Power-BI-Q-amp-A/ba-p/5173970>

Full retirement of Q&A by the end of December 2026, with users directed to Copilot.

29. Canner. WrenAI repository. <https://github.com/Canner/WrenAI>

Open-source GenBI agent with about 18,000 GitHub stars (October 2026); the nearest open-source neighbour to this project.

30. Hex. (2026). How Hex's Notebook Agent changed pricing analysis.
<https://hex.tech/blog/notebook-agent-pricing-analysis/>

The notebook-as-lineage mechanism that sets the bar any traceability claim must beat.

31. dbt Labs. (2026). State of analytics engineering (n=363, fielded December 2025 – February 2026). <https://www.getdbt.com/resources/state-of-analytics-engineering-2026>

Trust in data rising from 66% to 83% year over year; 71% cite hallucinated output reaching stakeholders as a top concern.

32. IBM. (2026, March 30). Data delivery delays are slowing decisions more than you think. <https://www.ibm.com/think/insights/data-access-delays-slowing-decisions>

Reports one-to-four-week enterprise data-request turnaround (attributed to Sigma Computing) and that 76% of businesses have decided without consulting data because

access was too difficult (attributed to Sisense). A citation chain rather than a primary study, and cited as such.

33. Anthropic. (2024, November 25). Introducing the Model Context Protocol.
<https://www.anthropic.com/news/model-context-protocol>

The announcement that dates the protocol's release, with reference servers including Postgres among the originals.

34. Linux Foundation. (2025, December 9). Linux Foundation announces the formation of the Agentic AI Foundation. <https://www.linuxfoundation.org/press/linux-foundation-announces-the-formation-of-the-agentic-ai-foundation>

Records the transfer of MCP to a neutral foundation co-founded by Anthropic, Block and OpenAI, with support from Google, Microsoft and AWS.

Chapter 2. Project Justification

The problem. An enterprise data analyst works through a steady queue of ad hoc business questions, and each one takes the same manual steps: find the source, read the schema, write and debug the query, reconcile results across sources, chart them, interpret them and check them. One industry report puts typical turnaround for a data request at one to four weeks, and cites a survey in which 76% of businesses said they had made a decision without consulting data because it was too hard to access [32]. When analysis is slow, organisations decide without it.

Why speed alone is not the problem. Every major BI vendor now offers natural-language querying, yet the evidence points to trust, not speed, as what limits adoption. In dbt Labs' 2026 survey of 363 data practitioners, the share who rate trust in data as important rose from 66% to 83% in a year, and 71% worry about hallucinated or incorrect data reaching stakeholders [31]. In a separate survey, 24.9% name accuracy and hallucination risk as their top concern about generative AI in analytics [23]. The most damaging errors are silent: in one study, 30.9% of text-to-SQL errors were queries that ran and returned a plausible but wrong result [4]. A fast answer that nobody is willing to stand behind saves the analyst no time.

Why now. Three things have changed in the last two years. Agents can sustain longer tasks: the task horizon METR measures has doubled about every seven months since 2019, and about every three months since 2024 [11][12]. Tool access is becoming standard: MCP, released in November 2024 [33] and now under the Linux Foundation's Agentic AI Foundation [34], makes connecting a new data source mostly a configuration task. And single text-to-SQL steps are strong enough to compose: the best agents reach 96.70% execution accuracy on Spider 2.0-Snow and 76.23% on the harder Lite setting [1]. Reliable multi-step analysis has not arrived; the best agent scores 14.55% on DABstep's hard tasks [6]. This project is therefore designed to make failures cheap to detect rather than to promise they will not happen.

Why not a better semantic layer. Incumbent products answer inside a scope curated before the question is asked: a semantic view, a Genie Agent of up to 50 tables, a Power BI semantic model or a set of Pulse metrics (see the State-of-the-Art Summary). Those scopes are widening, but a question that needs data outside them cannot be answered until someone adds it, and ad hoc questions are often exactly those. InsightPilot takes the other route. It builds the plan when the question is asked, from whatever sources are connected; checks each hop with an independent query; and records lineage so the analyst can inspect the one hop they doubt instead of re-deriving the whole answer. It uses a dbt semantic layer when one exists but does not require one.

Why it is academically worthwhile. The project depends on the weakest capability measured in the literature, multi-step analysis [6], and tests a claim that no prior system makes: that per-hop verification and hop-level lineage make checking an answer cheaper than deriving it (see the Literature Search). The major BI platforms do not publish product-level execution accuracy on public benchmarks (see the State-of-the-Art Summary). The closest research system, RADAR, measures silent errors on existing benchmarks [18]; this project adds a seeded-error set, so it can also report which planted errors each check catches. Publishing multi-hop execution accuracy, silent-error rate and verifier coverage together is a contribution whether or not the product succeeds, and it is the basis for a derivative paper.

Why it is worth two semesters. CMPE 295A delivers the measurement infrastructure and one narrow end-to-end path: the harness and question set, Postgres and CSV connectors, plan generation with an approval gate, fetch and analytics agents, lineage capture, hop verification and an exportable investigation record. CMPE 295B adds replanning, hop override, a REST source, semantic-layer consumption and a time-boxed modelling agent. Each semester ends with something that can be demonstrated and measured, and the first does not depend on the second.

Publication and funding potential. The planned derivative paper measures verification cost and silent-error rate for agentic BI across sources, reporting multi-hop execution accuracy, verifier coverage against the seeded-error set, and cost per investigation by retry count. Candidate venues are a data-management workshop on human-in-the-loop analytics, such as HILDA at ACM SIGMOD, or an agent-reliability workshop at a machine-learning conference, such as the ICML 2026 Workshop on Failure Modes in Agentic AI, where related work appeared [9]. The final venue will be chosen with the advisor. The working prototype, the published question set and the measured results make the project suitable for presentation at the SJSU Expo.

What this project does not claim. Our own estimate, built from assumed times for each step of a 90-minute multi-source question, puts the gain at about $2.9\times$ in work time (plausible range $1.8\times$ to $4.2\times$), not $10\times$, and at about $1.0\times$, no gain, for a familiar single-source question, where the analyst should keep writing SQL. The project makes no accuracy comparison with an incumbent product (see the State-of-the-Art Summary). Its riskiest assumption, that checking an answer you did not derive is cheaper than deriving it, is untested, and the verification-cost study in CMPE 295A (milestone M4) is designed to test it.

Chapter 3. Project Requirements

Actors and scope

Table 3. Actors

Actor	Description	Main goals
Data analyst (primary user)	Enterprise analyst who already writes SQL and absorbs an ad-hoc request queue	Reach a verified answer to a multi-source question faster than deriving it; retain control and the ability to override
Business stakeholder	Non-technical requester who consumes answers and cannot verify them	Get a plain-language answer with its provenance, clearly marked when not yet reviewed
Staff data scientist	Senior reviewer who audits method and lineage	Inspect method-level detail, override modelling choices, catch what the system could not
Analytics budget owner	Economic buyer; not a user	Provenance retained and queryable across investigations; adoption observable
Data platform lead	Security and governance gate; can veto without using the product	Read-only access, credentials inside the boundary, a readable audit log of every query issued
External data sources	Postgres, CSV/Excel files, REST JSON APIs, dbt semantic layers	Accessed read-only through connectors with inherited permissions
Model provider	Frontier LLM API reached with the deploying organisation's own key	Used for planning, term resolution and SQL generation only; never for verification

Use cases

Table 4. Use-case catalogue

ID	Use case	Primary actor	Summary
UC-01	Connect a data source	Analyst	Register a read-only Postgres connection or upload a CSV/Excel file; inferred schema shown for correction
UC-02	Ask a question	Analyst	Pose a business question inside an investigation that retains context and established facts
UC-03	Resolve ambiguous terms	System / Analyst	When a term maps to more than one column or grain, halt and ask rather than guess; confirmed bindings persist
UC-04	Review and approve a plan	Analyst	Inspect the generated analysis plan, edit it, and approve before anything executes
UC-05	Execute an investigation	System	Specialist agents run plan steps; each handoff emits structured output and lineage
UC-06	Verify a hop	System	Each step is checked by an independent query; failures surface immediately

ID	Use case	Primary actor	Summary
UC-07	Replan on invalidation	System	A failed check invalidates the remaining sub-plan, which is regenerated with the failure as context
UC-08	Inspect lineage	Analyst / Data scientist	Open any hop to see sources touched, statement issued, transform applied and row counts in and out
UC-09	Re-run or override a hop	Analyst	Re-execute one hop in isolation and diff against the stored result, or override it and re-flow downstream
UC-10	Export an investigation	Analyst	Produce one self-contained artifact that renders without the application running
UC-11	Sign off an answer	Analyst	Mark an investigation reviewed, by whom and when; unreviewed answers are gated on export
UC-12	Audit queries issued	Platform lead	Read an append-only log of every statement issued to every source, without the product running
UC-13	Measure accuracy	Team	Run the fixed question set and the seeded-error set through the harness and publish the results

Functional requirements

Each requirement has a priority. Essential (E) requirements must work by the end of CMPE 295B, Desired (D) requirements are planned and scheduled, and Optional (O) requirements are stretch goals.

Table 5. Functional requirements

ID	User story	Pri.	Trace	Acceptance criterion
FR-01	As a team, we want a fixed question set with ground-truth answers and an automated harness, so accuracy can be measured rather than asserted.	E	UC-13	30–50 questions execute unattended; execution accuracy reported per run and on every pull request
FR-02	As a team, we want a seeded-error set with planted wrong joins, grains and mappings, so silent errors can be counted.	E	UC-13	Each seeded defect is labelled; the harness reports which were caught and which escaped every check
FR-03	As an analyst, I want a read-only Postgres connection, so the system can query the warehouse without write access.	E	UC-01	Connection rejects any DDL or DML; schema introspection lists tables, columns, types and foreign keys
FR-04	As an analyst, I want to upload CSV or Excel files and see the inferred schema, so unmodelled sources are usable.	E	UC-01	File loads via DuckDB with no ETL step; inferred types shown and correctable before use
FR-05	As an analyst, I want the system to produce an ordered analysis plan over named sources, so the approach is explicit before execution.	E	UC-02	Plan is a typed DAG; static validation rejects cycles, unknown sources and unresolved columns

ID	User story	Pri.	Trace	Acceptance criterion
FR-06	As an analyst, I want to see, edit and approve the plan before anything runs, so errors are caught before they cost a query.	E	UC-04	No query executes before approval on first run against a source; edits persist into execution
FR-07	As an analyst, I want ambiguous terms intercepted rather than guessed, so I do not receive a plausible wrong answer.	E	UC-03	When top candidates fall within the margin or differ in grain, execution halts and candidates are shown with evidence
FR-08	As an analyst, I want confirmed term bindings stored and reused, so the same clarification is not requested twice.	E	UC-03	Binding persists per organisation with provenance; later questions reuse it; stale bindings flagged on schema change
FR-09	As an analyst, I want fetch and analytics agents that execute plan steps, so questions are actually answered.	E	UC-05	Joins, filters, aggregations and period comparison produce results with row counts and declared grain
FR-10	As an analyst, I want every handoff to record its lineage, so the answer arrives with a trace.	E	UC-05	Each hop records sources, exact statement, parameters, transform, grain and row counts in and out
FR-11	As an analyst, I want each hop verified by an independent query, so failures surface at the hop that caused them.	E	UC-06	Row-count, grain, coverage, null-rate and boundary-total checks run per applicable step; results stored with the hop
FR-12	As an analyst, I want to open a hop and read it as a step rather than a JSON blob, so verification takes seconds.	E	UC-08	Hop view shows statement, inputs, outputs, counts and check results on one screen
FR-13	As an analyst, I want to re-run a single hop in isolation and diff it, so I can confirm reproducibility.	E	UC-09	Re-run uses the recorded statement and parameters; differences are shown rather than silently overwritten
FR-14	As an analyst, I want to export a self-contained investigation artifact, so the result can be checked without the product.	E	UC-10	Single file renders in a browser or text editor with question, plan, hops, statements, counts and answer
FR-15	As an analyst, I want the system to name the hops it is least certain about, so doubt is visible.	E	UC-06	Doubt block lists least-certain hops with reasons; no confidence percentage is presented as accuracy
FR-16	As a platform lead, I want an append-only log of every query issued, so the system can be audited independently.	E	UC-12	JSONL file on disk readable without the application; records timestamp, source, statement and actor
FR-17	As an analyst, I want an investigation thread that inherits context, so follow-ups do not restate everything.	E	UC-02	Established facts, source scope and bindings carry into the next question in the thread

ID	User story	Pri.	Trace	Acceptance criterion
FR-18	As an analyst, I want the plan regenerated when a check fails, so a bad step does not propagate.	D	UC-07	Failed verification invalidates the remaining sub-DAG and triggers one bounded regeneration with the failure as context
FR-19	As an analyst, I want to override a hop and have downstream steps re-flow, so I can correct the system.	D	UC-09	Override persists as a binding; downstream steps re-execute, or the plan is invalidated and replanned if the grain changes
FR-20	As an analyst, I want a REST JSON connector, so a third source type is usable.	D	UC-01	Configurable endpoint and response shaping; results enter the plan as an ordinary source
FR-21	As an analyst, I want dbt semantic-layer definitions consumed when present, so existing correctness work is inherited.	D	UC-03	Where a semantic-layer definition exists for a term it takes precedence over any inferred binding
FR-22	As a stakeholder, I want unreviewed answers clearly marked and gated, so an unverified number does not reach a slide.	D	UC-11	Export of an unreviewed investigation requires explicit acknowledgement; sign-off records reviewer and time
FR-23	As an analyst, I want the same question to reuse its approved plan, so answers are reproducible.	D	UC-04	Plan cache keyed on normalised question, bindings, source set and schema version; a schema change misses the cache
FR-24	As a data scientist, I want a time-boxed modelling step with a model card, so predictions are reviewable.	D	UC-05	Model card states target definition, split strategy, features used and excluded with reasons, and an explicit not-attempted list
FR-25	As an analyst, I want a chart chosen and justified by data shape, so results are presentable.	D	UC-05	Chart type is named with the reason; a table is an acceptable fallback
FR-26	As an analyst, I want cost and latency previewed per plan, so expensive investigations are visible before running.	O	UC-04	Estimated token cost and wall-clock shown at the approval gate
FR-27	As a new user, I want to reach a first successful investigation quickly from a clean checkout.	O	UC-01	Docker Compose quickstart with seeded demo data; median clone-to-first-investigation under 30 minutes

Non-functional requirements

Table 6. Non-functional requirements

ID	Category	Requirement (measurable)	Pri.
NFR-01	Security	All source access is read-only; no DDL, DML or write-back is possible from any agent path	E
NFR-02	Security	Credentials remain inside the deployment boundary; the orchestration layer uses the deploying organisation's own frontier-model key and never logs secrets	E
NFR-03	Security	Row- and column-level permissions are inherited from the source and never re-implemented; access failures fail closed	E
NFR-04	Privacy	Columns flagged sensitive are excluded from value sampling and from model context; a schema-only mode is configurable	E
NFR-05	Auditability	100% of statements issued to any source appear in the append-only audit log, readable without the application running	E
NFR-06	Reliability	Verification runs as plain SQL against the source and never as a model call, so it cannot share the failure mode it checks	E
NFR-07	Reproducibility	A re-run of a recorded hop reproduces the stored result, or reports the difference; approved plans are reused for identical questions	E
NFR-08	Accuracy	Multi-hop execution accuracy on the fixed question set is measured and published. For context only, the best reported agent scores 14.55% on DABstep's hard tasks [6]; no comparison across task sets is claimed	E
NFR-09	Accuracy	At least 70% of seeded defects are rejected by the analyst with the trace, and at least twice the rate of a no-trace control	E
NFR-10	Usability	Verification cost ratio (minutes to accept or reject an answer divided by minutes to derive it) has a median below 0.5; a median above 0.8 is a declared stop trigger	E
NFR-11	Performance	The plan is presented within 15 seconds of the question for a six-hop plan on the reference schema	D
NFR-12	Cost	Cost per investigation is recorded and reported split by retry count, since expected attempts dominate the cost model	D
NFR-13	Portability	The full stack starts locally with one <code>docker compose up</code> ; median clone to first successful investigation under 30 minutes	D
NFR-14	Maintainability	The harness runs in CI on every pull request; back-end unit-test coverage at or above 70%	D
NFR-15	Compliance	The system produces answers only; it takes no autonomous action, sends no alerts that trigger workflows and makes no write-back	E

Constraints and assumptions

- All development and evaluation use public or synthetic datasets loaded into a local Postgres instance; no customer or confidential data is used at any point.
- Frontier-model usage will be funded by provider credits and paid team accounts, with a spend ceiling enforced per environment.
- The system requires a frontier-class model API. A fully local small-model deployment is out of scope, because tool-initialisation failure reaches an 89% error rate in a 3-billion-parameter model [10].
- The project is a research prototype evaluated offline; it is not offered as a service and makes no availability guarantee.
- The team has four members; scope is sized to one narrow end-to-end path plus the measurement infrastructure in CMPE 295A.

Chapter 4. Dependencies and Deliverables

Dependencies

Table 7. Showstopper dependencies (AK = Atharva Kulkarni, AN = Akshay Navani, PS = Pranjal Shrivastava, SZ = Shantanu Zadbuke)

ID	Dependency	Why it is a showstopper	Mitigation	Owner
D-01	Frontier-model API access and budget	Planning, term resolution and SQL generation need a capable model: tool-initialisation failure reaches 89% in a 3B model, and the same study puts the minimum viable size at about 14B parameters [10]	Provider-agnostic client so a second provider, or a locally hosted model of 32B parameters or more, can be substituted; verification is plain SQL and uses no model tokens, which caps spend	AK
D-02	A realistic Postgres schema with a public or synthetic dataset	Every accuracy figure depends on a fixed question set over a known schema; a toy schema would flatter the system, because accuracy drops sharply on realistic benchmarks (see the Literature Search)	Generate a synthetic dataset locally (seeded orders, shipments and an unmodelled carrier CSV), so no real company data is needed and nothing external is required once it is loaded	AN
D-03	Ground-truth answers for the fixed question set	Without ground truth there is no execution-accuracy measurement, and that measurement is the project's primary publishable result	Authored by the team alongside the question set in Sprints 1 and 2; scoped work rather than an external dependency, but the project stalls without it	PS
D-04	A reproducible environment every member can run	With four people working in parallel, anyone whose environment does not run the full stack is blocked	Docker Compose from the first sprint so the environment is reproducible rather than hand-configured	AN
D-05	Analyst participants for the verification-cost study	The riskiest claim, that checking an answer is cheaper than re-deriving it, can only be measured with people, and milestone M4 depends on it	Recruit five analysts from the SJSU MS cohort and professional networks; the task needs one hour per participant and no software	SZ
D-06	Advisor approval of scope and evaluation plan	The advisor grades both semesters; if the scope or the evaluation design is not agreed early, a sprint's work may not count toward the deliverables	Review requirements, architecture and the measurement plan with Prof. Eranti in October 2026	All

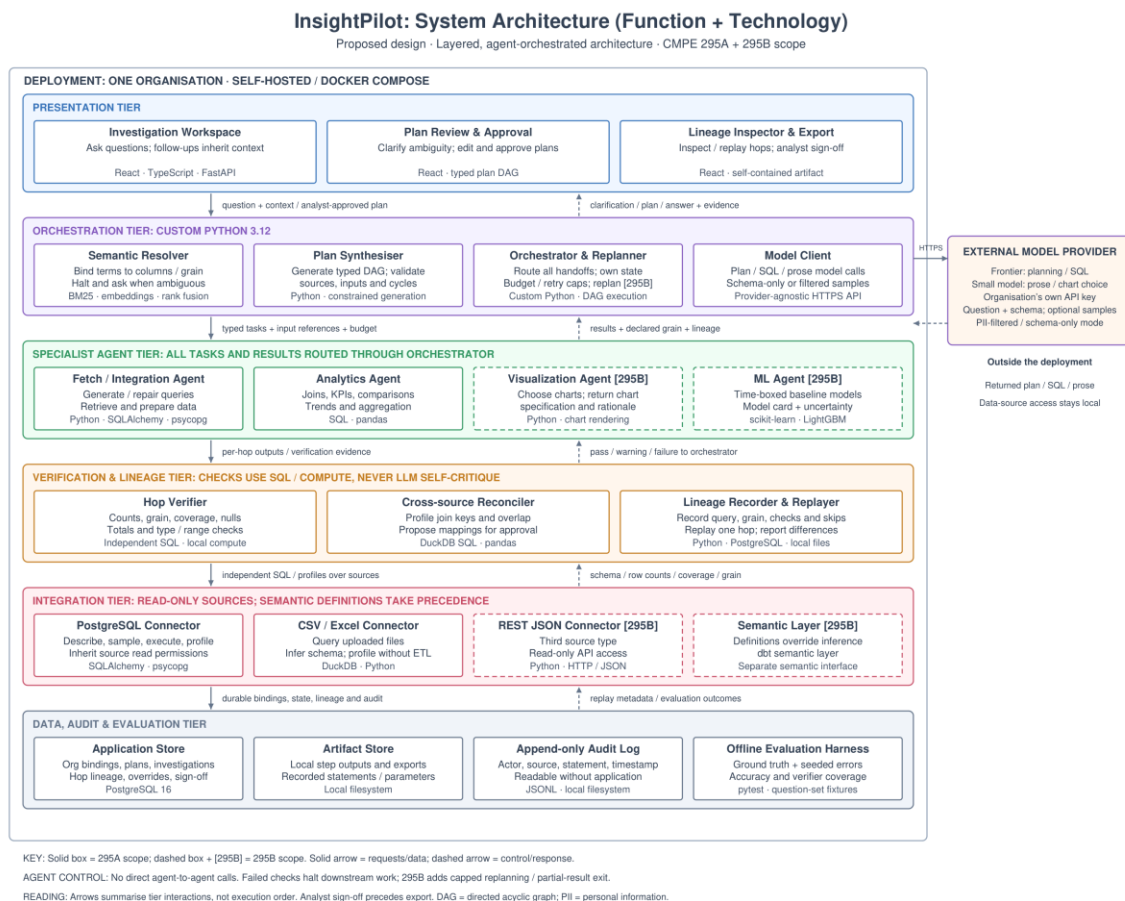
Deliverables

Table 8. Project deliverables

#	Deliverable	Description
1	Evaluation harness	Automated runner scoring multi-hop execution accuracy over a fixed question set, executed in CI on every pull request
2	Fixed question set with ground truth	30–50 questions over a known schema with authored expected results, published alongside the code
3	Seeded-error test set	Investigations with deliberately planted wrong joins, grains and mappings, used to measure silent-error rate
4	Connectors	Read-only Postgres connector and CSV/Excel ingestion through DuckDB, with inferred schema correction
5	Orchestrator and plan surface	Typed plan DAG with static validation, plus the review and approval gate shown before execution
6	Specialist agents	Data fetch/integration and analytics agents emitting structured output and lineage per handoff (visualization agent in CMPE 295B)
7	Verification layer	Hop verifier with row-count, grain, coverage, null-rate and boundary-total checks, and the cross-source reconciler
8	Lineage store and inspector	Hop-level records with content-addressed step outputs, single-hop replay, and the readable hop view
9	Exportable investigation artifact	One self-contained file rendering without the application, carrying question, plan, hops, statements, counts and sign-off
10	Audit log	Append-only JSONL record of every statement issued to every source, readable without the product running
11	Published measurements	Multi-hop execution accuracy, silent-error rate and verifier coverage, reported with method and failure cases
12	Verification-cost study result	The timed task with five analysts, reported even if the result is negative
13	Derivative research paper	Paper on measuring verification cost and silent-error rate for cross-source agentic BI, submitted to a peer-reviewed workshop or conference, subject to advisor agreement
14	Documentation and demo	Architecture and setup documentation, a quickstart, a demo video and the Expo prototype
15	Early prototype demo	A vibecoded walkthrough of the agentic flow on a relational database source, requested by the advisor; target date to be confirmed with the advisor

Chapter 5. Project Architecture

Figure 1. System architecture showing each component's function and technology



Architectural style. InsightPilot uses a layered architecture of six tiers, deployed for a single organisation. Generation and verification are kept apart: components that produce an answer may call a language model, and components that check an answer may not. Checks run as SQL or local computation, so they cannot share the failure mode of the model whose output they check. Two rules apply across the figure. All tasks and results pass through the orchestrator, with no direct agent-to-agent calls, so every handoff is observable, budgeted and recorded. And the arrows show which tiers interact, not the order of execution: each hop is verified as soon as it completes, not once at the end of the plan.

Presentation tier (React, TypeScript, FastAPI). The investigation workspace is where the analyst asks a question; follow-up questions inherit its context. Plan review and approval is where ambiguous terms are clarified and the plan is edited and approved before anything runs, so a wrong plan costs one edit rather than a wrong answer. The

lineage inspector replays individual hops and exports the signed-off investigation as one self-contained file.

Orchestration tier (Python 3.12). The semantic resolver maps business terms to columns and grain, combining BM25 keyword search over column names and comments with embedding search and with matches against sampled values. When candidates score too closely or disagree on grain, it stops and asks instead of guessing, because a wrong guess produces the silent semantic errors described in the Literature Search [4]. The plan synthesiser uses constrained generation to produce a typed plan, a directed acyclic graph of steps, and validates its sources, inputs and dependencies before execution. Writing the orchestrator ourselves, rather than using an agent framework, keeps the plan an explicit data structure that can be shown, edited, approved and cached. The orchestrator dispatches every task, holds execution state, and enforces budgets and retry limits; replanning after a failed check is CMPE 295B scope. The model client works with any provider and can send the schema only, or a filtered sample of values.

Specialist agent tier. The fetch and integration agent writes and repairs queries and prepares data. The analytics agent performs joins, aggregations, KPIs, trends and comparisons in SQL and pandas. Two agents are planned for CMPE 295B: a visualization agent that returns a chart specification with its rationale, and a machine-learning agent that builds time-boxed baseline models with a model card and an uncertainty statement.

Verification and lineage tier. The hop verifier checks row counts, grain, coverage, nulls, totals and value ranges on each hop with independent SQL. The cross-source reconciler profiles join keys and value overlap and proposes a mapping for the analyst to approve, since mapping one region code to another is a business decision. The lineage recorder stores each hop's query, grain and check results, including checks that were skipped, and can replay a single hop and report differences. A failed check stops downstream work.

Integration tier (read-only). The PostgreSQL connector describes, samples, queries and profiles a database using the source's own read permissions. The CSV and Excel connector queries uploaded files through DuckDB, inferring and profiling their schema without an ETL step. A REST JSON connector and dbt semantic-layer support follow in CMPE 295B; semantic-layer definitions, when present, override inferred bindings.

Data, audit and evaluation tier. PostgreSQL 16 stores bindings, plans, investigations, hop lineage, overrides and sign-off. Recorded statements, step outputs and exports are kept on the local filesystem. An append-only JSONL audit log records the actor, source, statement and time of every query, and can be read without the application. The offline evaluation harness runs the question set and the seeded-error set under pytest to produce the published accuracy and verifier-coverage figures.

External model provider. Only model calls leave the deployment. A frontier model handles planning and SQL generation, and a smaller model writes prose (and, in CMPE 295B, chooses charts), using the deploying organisation's own API key. Requests carry the question and schema, with optional samples; a PII-filtered or schema-only mode is available. Data-source access never leaves the deployment.

One investigation, end to end. The analyst asks a question that spans a warehouse table and an uploaded CSV. The resolver asks once which of two columns means "region", and the synthesiser produces a plan that passes validation. The analyst narrows one fetch and approves the plan. The orchestrator sends the first task to the fetch agent, and the verifier checks its counts, grain and coverage before the next task starts. The coverage check finds 8% of region codes unmatched; the analyst accepts the proposed mapping, which is saved as a binding. The analytics agent then joins and aggregates, with each hop verified. The answer arrives with a note naming the least certain hop, and after sign-off the investigation is exported as one file.

Deployment. Each organisation runs its own copy with Docker Compose, which is also the basis of the 30-minute quickstart. There is no shared multi-tenant service.

Chapter 6. Project Design

To be completed in Workbook Assignment 2 (UML class and sequence diagrams for the plan DAG and the verification path, UI mockups for the investigation surface, plan review and lineage inspector, the database ER diagram for bindings and investigation records, and the plan and lineage schemas).

Chapter 7. QA, Performance, Deployment Plan

To be completed in Workbook Assignment 2 (test strategy, the harness as a continuous evaluation loop, the seeded-error protocol and verifier-coverage measurement, performance budgets per loop beat, cost benchmarking by retry count, and the deployment pipeline).

Chapter 8. Implementation Plan and Progress

Implementation starts with measurement. The team builds the evaluation harness and the fixed question set before any agent, then gets one narrow path working end to end: connect a source, plan a question, run two hops, verify them, record lineage and export the result. Each component is deepened only after that path works, because a system with several agents and no measurement cannot show whether it works. The early prototype demo requested by the advisor (deliverable 15) is a walkthrough of this first path on a relational database source.

Programming and Execution Environment

Table 9. Programming and execution environment

Layer	Technology (version)	Purpose	Owner	Status
Source control and CI	GitHub repository, GitHub Actions	Code hosting, pull-request review, harness run on every pull request	AN	Repository created; CI in Sprint 1
Language and runtime	Python 3.12	All orchestration, agent and verification code	AK	Installed and verified by all four members
Orchestrator	Custom typed plan DAG (no agent framework)	The plan must be validatable, diffable and cacheable	AK	Sprint 1 spike
Model access	Provider-agnostic client; project API key	Planning, term resolution and SQL generation	AK	Sprint 1 spike
Database access	SQLAlchemy, psycopg	Read-only Postgres introspection and execution	AN	Sprint 1
File and CSV engine	DuckDB	Query CSV, Excel and Parquet directly with no ETL step	AN	Sprint 1
Profiling	DuckDB SQL, pandas	Cardinality, uniqueness, overlap and null-rate profiling	PS	Sprint 2
Application store	PostgreSQL 16	Bindings, investigation records, sign-off state	AN	Sprint 1
Audit log	Append-only JSONL on disk	Readable without the application running	SZ	Sprint 2
API and front end	FastAPI, React, TypeScript	Investigation surface, plan review, lineage inspector	SZ	Sprint 2
Modelling (295B)	scikit-learn, LightGBM	Time-boxed baseline models within a minutes budget	PS	CMPE 295B
Packaging	Docker, Docker Compose	Reproducible local stack; later the quickstart	AN	Sprint 1

Layer	Technology (version)	Purpose	Owner	Status
Harness	pytest with a question-set fixture	Execution-accuracy and seeded-error measurement	PS	Sprint 1

Development Tools per Team Member

Table 10. Development tools acquired per member (all installed and verified)

Tool	Atharva Kulkarni	Akshay Navani	Pranjal Shrivastava	Shantanu Zadbuke
IDE with Python, TypeScript and Docker extensions	Yes	Yes	Yes	Yes
Python 3.12 toolchain and pytest	Yes	Yes	Yes	Yes
Docker Desktop and Docker Compose	Yes	Yes	Yes	Yes
PostgreSQL 16 locally or in Docker	Yes	Yes	Yes	Yes
DuckDB CLI and Python package	Yes	Yes	Yes	Yes
Frontier-model API key through the shared project account	Yes	Yes	Yes	Yes
Node and the front-end toolchain	Yes	Yes	Yes	Yes
Database client for schema inspection	Yes	Yes	Yes	Yes
GitHub Projects board for task tracking	Yes	Yes	Yes	Yes

Prototyping and Simulation Environment

The project is software-only, so the prototyping environment is a reproducible simulated analytics estate rather than hardware:

- **Local stack.** Docker Compose running PostgreSQL, the application services and the harness, so all four members run an identical environment.
- **Reference schema.** A synthetic dataset generated by the team and loaded into local Postgres, seeding orders, shipments, customers and regions across roughly 40 tables. It is wide enough to be realistic, because accuracy drops sharply on realistic schemas (see the Literature Search) and a narrow schema would flatter the system.
- **Unmodelled source.** A CSV file deliberately not represented in the warehouse schema, queried through DuckDB, so the evaluation always includes a cross-source question.

- **Fixed question set.** 30–50 questions with authored ground truth covering single-source, cross-source and ambiguous-term cases.
- **Seeded-error set.** The same investigations with deliberately planted wrong joins, grains and region mappings, used to measure what escapes every check.

Sample Programs and Technology Spikes

Table 11. Sample programs used to learn each technology (owner initials as in Table 7)

Spike	Goal and what it proves	Owner	Target
SP-1 Postgres read path	Connect read-only, introspect the schema, execute a parameterised query and return row counts and grain	AN	Oct 11, 2026
SP-2 DuckDB file path	Query a CSV directly with SQL, infer and correct types, and join it to a Postgres result	AN	Oct 11, 2026
SP-3 Typed plan object	One model call returns a plan that parses into the typed DAG and passes static validation; an invalid plan is rejected with field-level errors	AK	Oct 11, 2026
SP-4 Hop verification	Run row-count, grain and coverage checks against a deliberately broken join and confirm the failure is caught and reported	PS	Oct 25, 2026
SP-5 Lineage record and replay	Record a hop, re-execute it from the record alone, and diff the result	SZ	Nov 8, 2026
SP-6 Harness run	Execute five questions end to end with ground-truth comparison and print an accuracy figure	PS	Nov 22, 2026

Prototype Implementation Plan

Table 12. Prototype increments for CMPE 295A

Step	Increment	Technologies exercised	Sprint
1	Measurement first: repository, CI, harness skeleton, fixed question set with ground truth, seeded-error set	GitHub Actions, pytest, PostgreSQL	S1
2	Source substrate: read-only Postgres connector and CSV/Excel ingestion with inferred schema	SQLAlchemy, psycopg, DuckDB	S1–S2
3	Plan path: question to typed plan DAG, static validation, plan shown and approved	Provider-agnostic model client, Pydantic	S2–S3
4	Execute path: fetch and analytics agents run approved steps and emit structured output	Python agents, SQL	S3
5	Verify path: hop verifier checks each step; failures surface at the hop that caused them	SQL checks, DuckDB	S3–S4

Step	Increment	Technologies exercised	Sprint
6	Lineage path: hop records, readable hop view, single-hop re-run and diff	PostgreSQL, FastAPI, React	S4
7	Export and audit: self-contained investigation artifact and append-only audit log	JSONL, templating	S4–S5
8	Grounding quality: ambiguity interception, binding store, doubt surfacing	BM25, embeddings, reciprocal rank fusion	S5
9	Measure and report: execution accuracy, silent-error rate, verifier coverage, cost per investigation by retry count	Harness, seeded-error set	S5–S6

Progress Log

Table 13. Implementation progress

Date	Item	Status
Sep 14, 2026	Project repository created with README, concept and roadmap	Done
Sep 2026	Project concept and abstract agreed with the advisor	Done
Sep 2026	Python 3.12 environment and dependencies installed and verified by all four members	Done
Sep–Oct 2026	Literature search, state of the art, requirements and architecture (this workbook)	Done
Sep 28 – Oct 11, 2026	Sprint 1: repository, CI, Docker Compose, harness skeleton, question set, spikes SP-1 to SP-3	In progress
Oct 2026	Design being finalised; spike started on the agentic flow for a relational database source	In progress
Oct 12 – Oct 25, 2026	Sprint 2: plan synthesis, CSV ingestion, ground truth and seeded errors, spike SP-4	Planned

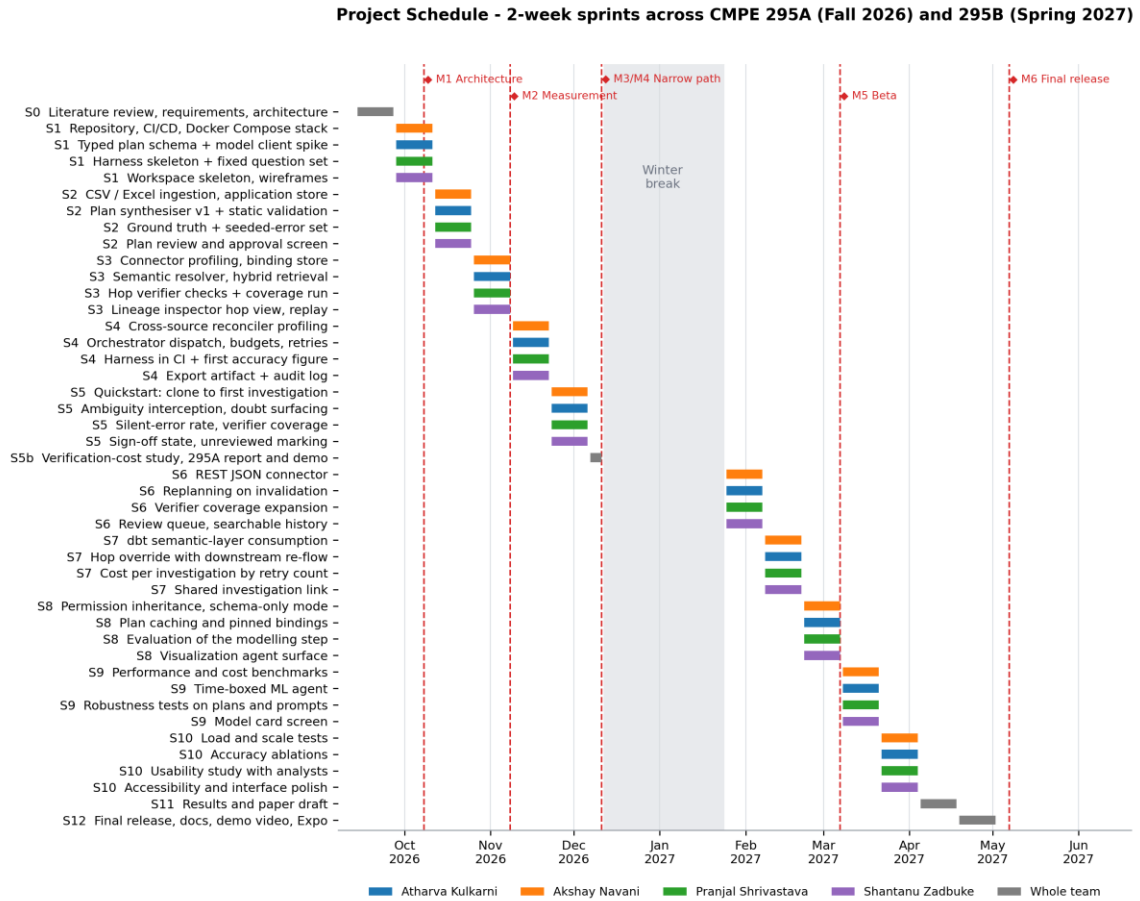
Chapter 9. Project Schedule

The project runs in two-week sprints from 14 September 2026, when the project repository was created, through the end of CMPE 295B, and work is tracked on a GitHub Projects board. At the end of each sprint the team demonstrates what was merged and updates the progress log in Chapter 8. In every sprint each member owns a separate stream of work, so tasks do not overlap; the last two days of a sprint are reserved for joint integration. The early prototype demo requested by the advisor (deliverable 15) will be added to the schedule once its date is agreed.

Milestones

Table 14. Milestones and exit criteria

Milestone	Date	Exit criteria
M1 Requirements and architecture baseline	Oct 8, 2026	Literature search, requirements, architecture and schedule complete and reviewed with the advisor; Workbook 1 submitted
M2 Measurement baseline	Nov 8, 2026	Harness skeleton, fixed question set with ground truth, and seeded-error set complete; hop-check spike (SP-4) passes against a deliberately broken join
M3 Narrow path end to end	Dec 6, 2026	One question spanning Postgres and an uploaded CSV is planned, approved, executed, verified per hop and exported as a self-contained artifact
M4 Evidence for the central claim	Dec 11, 2026	Verification-cost study run with five analysts; silent-error rate and verifier coverage reported even if the result is negative
M5 Beta	Mar 7, 2027	Replanning, hop override, REST connector, dbt consumption, sign-off and export working on the staging environment
M6 Final release	May 7, 2027	Hardened system, published accuracy and coverage figures, paper draft, demo video and Expo prototype

Figure 2. Project schedule by sprint and team member

Sprint plan and task assignment

Table 15. Task assignment per sprint

Sprint (dates)	Atharva Kulkarni	Akshay Navani	Pranjal Shrivastava	Shantanu Zadbuke
S0 (Sep 14 – Sep 27)	Agent and planning literature; requirements	Text-to-SQL literature; architecture	Reliability and evaluation literature; dependencies	State of the art; justification; screen sketches
S1 (Sep 28 – Oct 11)	Typed plan schema; model client spike (SP-3)	Repository, CI, Docker Compose; Postgres connector (SP-1); DuckDB path (SP-2)	Harness skeleton; fixed question set drafted	Investigation workspace skeleton; wireframes
S2 (Oct 12 – Oct 25)	Plan synthesiser v1 with static validation	CSV and Excel ingestion; application store schema	Ground-truth answers; seeded-error set; hop-check spike (SP-4)	Plan review and approval screen

Sprint (dates)	Atharva Kulkarni	Akshay Navani	Pranjal Shrivastava	Shantanu Zadbukey
S3 (Oct 26 – Nov 8)	Semantic resolver: hybrid retrieval and margin threshold	Connector profiling; binding store persistence	Hop verifier checks; first verifier-coverage run	Lineage inspector hop view; replay spike (SP-5)
S4 (Nov 9 – Nov 22)	Orchestrator dispatch; budgets and retry caps	Cross-source reconciler profiling	Harness in CI; first published accuracy figure (SP-6)	Export artifact; append-only audit log
S5 (Nov 23 – Dec 6)	Ambiguity interception; doubt surfacing	Quickstart: clone to first investigation under 30 minutes	Silent-error measurement; verifier coverage reported	Sign-off state; unreviewed-answer marking
Dec 7 – Dec 11	Verification-cost study, five analysts (joint)	295A report and demo (joint)	Results write-up (joint)	Demo preparation (joint)
Winter break (Dec 12 – Jan 24)	Design notes for Workbook 2 (optional)	Deployment notes (optional)	Seeded-error expansion (optional)	Screen mockups v2 (optional)
S6 (Jan 25 – Feb 7, 2027)	Replanning on invalidation	REST JSON connector	Verifier coverage expansion	Review queue; searchable history
S7 (Feb 8 – Feb 21)	Hop override with downstream re-flow	dbt semantic-layer consumption	Cost per investigation by retry count	Shared investigation link
S8 (Feb 22 – Mar 7)	Plan caching and pinned bindings	Permission inheritance; schema-only mode	Evaluation of the modelling step	Visualization agent surface
S9 (Mar 8 – Mar 21)	Time-boxed ML agent	Performance and cost benchmarks	Robustness tests on plans and prompts	Model card screen
S10 (Mar 22 – Apr 4)	Accuracy ablations	Load and scale tests	Usability study with analysts	Accessibility and interface polish
S11 (Apr 5 – Apr 18)	Paper: method and results	Paper: systems and cost	Paper: evaluation and coverage	Paper: figures and user study
S12 (Apr 19 – May 2)	Final hardening	Release packaging and runbooks	Final measurement run	Demo video and Expo booth